ShopFlow

Design and build an online store with flash sales

Design and build, on your own, the backend of an online store where customers buy products held in several warehouses. Stock is limited, and during flash sales thousands of customers compete for a few units at the same moment. The system must never sell stock it doesn't have, never get money wrong, and keep working under load and when parts fail. You make every design decision, and you are expected to explain and defend each one.

Functional requirements

- **Catalog:** products belong to categories, and each category has its own set of product attributes (for example, size for clothing and voltage for electronics). Staff can add new attributes without a redeploy. Anyone can browse, filter and search the catalog.
- **Stock:** each warehouse holds its own stock of each product. Staff can adjust stock, and every adjustment is audited.
- **Orders:** customers order several products at once. Stock is reserved until payment; unpaid orders expire after 15 minutes and free their stock. Customers can cancel an order until it ships.
- **Payments:** payments go through an external payment provider, which you also build as a fake that can be slow, fail, or lose its replies. Support staff can issue full or partial refunds. Every money movement is recorded and reconciled every day.
- **Flash sales:** a limited quantity is sold at a special price within a time window, with a limit per customer.
- **Recalls:** when a product is recalled, every affected order that hasn't been delivered is refunded automatically.
- **Shipping:** an order can ship in several parcels, one from each warehouse involved. Carrier updates may arrive twice or in the wrong order.
- **History and notifications:** each order has a full timeline, and each customer has an activity feed covering the last 90 days. Customers are notified of key events and can turn off the optional ones.
- **Users:** customers, catalog managers (each limited to their own categories) and support agents. Everyone sees only what their role allows.

Non-functional requirements

- **Correctness:**
  - Stock is never oversold, even with 1,000 buyers competing for 10 units.
  - No order is charged twice or refunded twice.
  - An accepted order is never lost.
- **Scale:**
  - 100,000 products and millions of history events.
  - At 200 requests per second, 95% of reads complete within 200 ms and 95% of writes within 400 ms.

- **Resilience:** the system keeps working when any single instance, the message broker, the cache or the payment provider fails or slows down, and it recovers by itself.
- **Security:** login through an identity provider, rate limiting, and no personal data or secrets in logs.
- **Operability:**
  - Metrics, dashboards, alerts, centralized logs and end-to-end tracing.
  - Deployments with zero downtime.

Required technologies

Must use	Java 21+, Spring Boot, Maven **PostgreSQL, MongoDB, Oracle** (each must own real data) Kafka, Redis, OpenFeign, Spring Cloud Gateway Prometheus, Grafana, Splunk GitHub Actions, Jenkins, GitLab CI, Bitbucket Pipelines Kubernetes, OpenShift
Must NOT use	Cassandra
Out of scope	User interface (APIs only) and AI features.

How and where you use each technology is part of your design.

What to deliver

- A design document with architecture diagrams, your data model, API and message contracts, and a decision record for every significant choice.
- Working code in your own repository, with automated tests.
- Evidence that the non-functional requirements are met: load test results and failure experiments.
- A README from which anyone can run the system, plus a short demo.
- A design review with the other developer, run as a system design interview.